



PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PROGRAMMING WITH PYTHON

Exception Handling, File I/O, Modules and Packages

File I/O Operations - JSON

Lekha A

Computer Applications



PROGRAMMING WITH PYTHON

JSON

- JSON stands for JavaScript Object notation and is an open standard human readable data format.
- JSON is written as key and value pair.
- An empty JSON file simply contains two curly braces {}.
- Multiple value statements in JSON are separated by a comma sign.



PROGRAMMING WITH PYTHON

JSON

- The Syntax rules for JSON is given below
 - The data is simply a name value pair
 - Data/Object/arrays are separated by comma
 - Curly braces hold object
 - Square holds array



PROGRAMMING WITH PYTHON

JSON

- **JSON with {}**
 - It is stored as an object.
 - Key value pairs are used.
 - The program reads it like a dictionary.



PROGRAMMING WITH PYTHON

JSON

- Example
 - { "name": "Alpha", "age": 21 }
- It can be accessed as **data["name"], data["age"]**
- Or
- **for k, v in data.items():**
- **print(k, v)**

```
▼ root
  name "Alpha"
  age 21
```



PROGRAMMING WITH PYTHON

JSON

- **JSON with []**
 - It is stored as an array.
 - Multiple objects are kept inside.
 - The program reads it like a list.



PROGRAMMING WITH PYTHON

JSON

- Example
 - [{"name": "Alpha", "age": 21},
 - {"name": "Beta", "age": 20},
 - {"name " : " Gamma", "age": 18}]
- It can be accessed as
 - data[0]["name"], data[0]["age"]

▼ root [] 3 items

▶ 0

▶ 1

▶ 2

▼ root [] 3 items

▼ 0

name "Alpha"

age 21

▼ 1

name "Beta"

age 20

▼ 2

name "Gamma"

age 18



PROGRAMMING WITH PYTHON

JSON

- It can be accessed as
- **for entry in data:**
- **`print(entry["name"], entry["age"])`**
- Or
 - **`names = [entry["name"] for entry in data]`**



PROGRAMMING WITH PYTHON

JSON

- A JSON file can start with {} and still contain multiple dictionaries, but they must be stored as **keyed dictionaries inside one main dictionary**, not as a list of separate objects.



PROGRAMMING WITH PYTHON

JSON

- JSON allows only **one top-level value**, and that top-level value can be
 - one object (starts with `{}`) or
 - one array (starts with `[]`)
- If it starts with `{}`, everything inside must be structured as **key value pairs**.
- Multiple dictionaries must be placed under different keys.



PROGRAMMING WITH PYTHON

JSON

- Example
 - {
 - "alpha": {"age": 21, "city": ["Mangalore", "Bantwal"]} },
 - "beta": {"age": 20, "city": "Mysore"},
 - "gamma": {"age": 18, "city": "Bangalore"}
 - }

```
▼ root
  ▶ alpha
  ▶ beta
  ▶ gamma

▼ root
  ▼ alpha
    age 21
    ▼ city [] 2 items
      0 "Mangalore"
      1 "Bantwal"
  ▼ beta
    age 20
    city "Mysore"
  ▼ gamma
    age 18
    city "Bangalore"
```



PROGRAMMING WITH PYTHON

JSON

- Code can access the data
 - `data["emp1"]["salary"]`
- Or
 - for key in data:
 - `entry = data[key]`
 - `print(entry["name"], entry["salary"])`



PROGRAMMING WITH PYTHON

JSON

- In Python, JSON exists as a string.
- Example
 - `p = '{"name": "Bob", "languages": ["Python", "Java"]}'`
- It's also common to store a JSON object in a file.



PROGRAMMING WITH PYTHON

JSON

- `json.load()`
 - Reads JSON data from a file-like object and deserializes it into a corresponding Python object.
- `json.loads()`
 - Deserializes a JSON-formatted string into a corresponding Python object.



PROGRAMMING WITH PYTHON

JSON

- `json.dump()`
 - Serializes a Python object and writes it as JSON data to a file-like object.
- `json.dumps()`
 - Serializes a Python object into a JSON-formatted string.



PROGRAMMING WITH PYTHON

JSON

- Parse a JSON string

```
import json  
person = '{"name": "Bob", "languages": ["English", "French"]}'  
person_dict = json.loads(person)
```

```
print( person_dict)  
print(person_dict['languages'])
```

```
{'name': 'Bob', 'languages': ['English', 'French']}  
['English', 'French']
```



PROGRAMMING WITH PYTHON

JSON

- Read a file containing JSON object and parse the data into a dictionary called data in the example

```
import json
```

```
with open('name.json', 'r') as f:  
    data = json.load(f)
```

```
print(data)
```

```
{'name': 'Peter', 'languages': ['English', 'French']}
```



PROGRAMMING WITH PYTHON

JSON

```
import json
```

```
# Open the json file in 'read' mode
```

```
with open("emp.json","r",encoding='utf-8') as file:  
    jsonData = json.load(file)
```

```
print("Type of JSON Object: ", type(jsonData))
```

```
# Traversing the json file
```

```
for name in jsonData:  
    print("Name: ", name)  
    print("Phone Number: ", jsonData[name]["number"])  
    print("Age: ", jsonData[name]["age"])
```

```
print("Address:")
```

```
for line in jsonData[name]["address"]:  
    print(line)  
print()
```

Type of JSON Object: <class 'dict'>

Name: Barty

Phone Number: +61 2 3617 9451

Age: 50

Address:

10/365 Pacific Highway, Hornsby

Sydney, New South Wales

Australia - 2077.

Name: Molly

Phone Number: +91 9972354015

Age: 45

Address:

Ottery St Catchpole

Devon

London - 4567



PROGRAMMING WITH PYTHON

JSON

- Convert the dictionary to JSON

```
import json  
  
name_dict = {'name': 'Lupin',  
             'age': 33,  
             'children': None  
}
```

```
name_json = json.dumps(name_dict)
```

```
print(name_json)
```

```
{"name": "Lupin", "age": 33, "children": null}
```



PROGRAMMING WITH PYTHON

JSON

- Write JSON to a file in Python

```
import json

name_dict = {"name": "Andromeda",
"languages": ["English", "French"],
"married": True,
"age": 32
}
```



name
Text Document
85 bytes

```
with open('name.txt', 'w') as json_file:
    json.dump(name_dict, json_file)
```

```
{"name": "Andromeda", "languages": ["English", "French"], "married": true, "age": 32}
```



PROGRAMMING WITH PYTHON

JSON

- Pass additional parameters `indent` and `sort_keys` to `json.dumps()` and `json.dump()` method.

```
import json
person_string = '{"name": "Dennis", "languages": "English", "numbers": [2, 1.6, null]}'

person_dict = json.loads(person_string)

print(json.dumps(person_dict, indent = 4, sort_keys=True))

{
    "languages": "English",
    "name": "Dennis",
    "numbers": [
        2,
        1.6,
        null
    ]
}
```



PROGRAMMING WITH PYTHON

JSON

- Count the lines to store the students' addresses. Store these values in a list with the person's name and print the

```
import json
with open("emp.json","r") as file:
    jsonData = json.load(file)
```

```
result = []
for name in jsonData:
    data = [name, len(jsonData[name]["address"])]
    result.append( data )
print("Person vs Number of Address Lines:")
for name, lines in result:
    print( name + " ",end=" ")
    print(lines)
```

```
Person vs Number of Address Lines:
Barty  3
Molly  4
```



PROGRAMMING WITH PYTHON

JSON

- Store only the mappings between the students' names and their phone numbers.

```
import json
with open("emp.json", "r") as file:
    fileData = file.read()
    jsonData = json.loads(fileData)
```

```
phoneRecord = {}
for name in jsonData:
    phoneRecord[name] = jsonData[name]["number"]
with open("output.json", "w") as outputFile:
    json.dump(phoneRecord, outputFile)
print("Process Complete!")
```

Process Complete!



PROGRAMMING WITH PYTHON

Recap

- JSON files
- Syntax
- Reading a JSON file
- Writing a JSON file
- Converting Dictionary to JSON



THANK YOU

Lekha A

Computer Applications